



WeCloudData

Streamlit





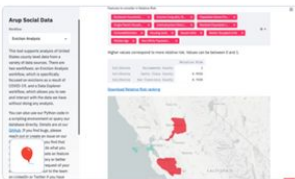
Introduction

Setting Up Streamlit & First App

Streamlit is an open-source Python library that enables data scientists and developers to create interactive web applications for machine learning and data science projects with minimal coding.

It provides a simple and intuitive interface for creating data-driven applications that can be easily shared and deployed on the web.

A faster way to build and share data apps



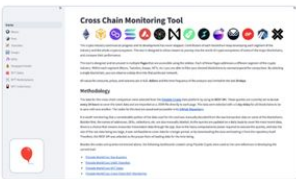
Arup Social Data

This is a repository for collection and analysis of open social data. This type of data can be useful to planners, NGO's,



Background Remover

A simple Streamlit app that removes the background on an uploaded image and lets you download the result.



Cross Chain Monitoring Tool

This app lets you compare +10 blockchains in different sectors and view the performance of each blockchain.



Dashboarding tools

Setting Up Streamlit & First App

Streamlit was founded by Adrien Treuille, Thiago Teixeira, and Amanda Kelly in 2019. The team had previously worked at Google X. After leaving Google, they saw a need for a tool that would allow data scientists and developers to create interactive applications more easily.

Streamlit was officially launched in late 2019 and quickly gained popularity in the data science community. Its ease of use, flexibility, and ability to create interactive applications quickly made it a favorite among developers.

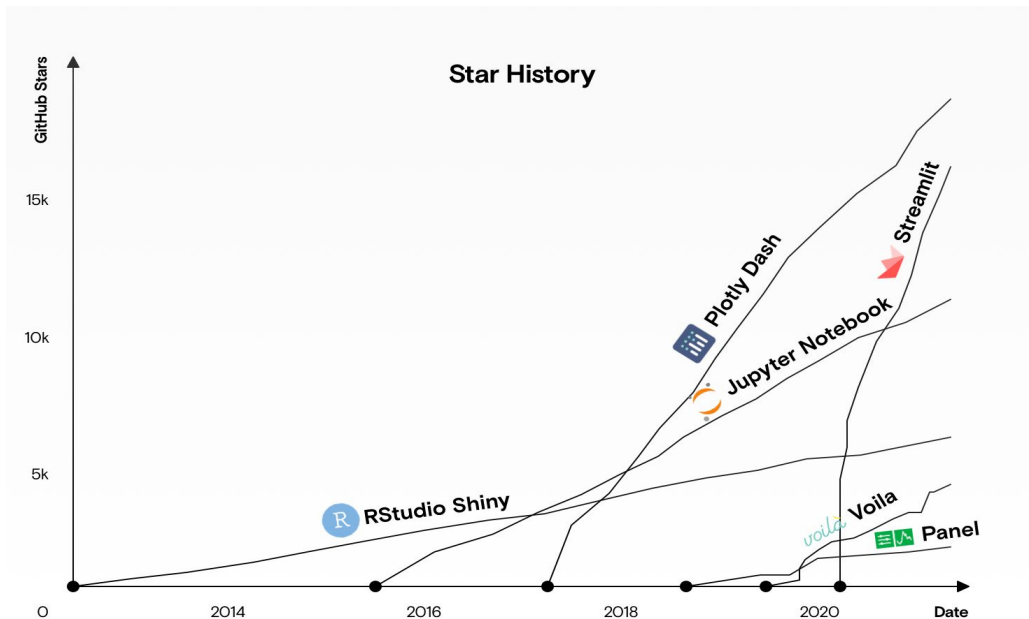
Since its launch, Streamlit has received multiple rounds of funding and has grown its team to include engineers, designers, and developers from around the world.

Snowflake acquired Streamlit for \$800M to help customers build data apps in 2022



Dashboarding tools

Setting Up Streamlit & First App



Analyze: Manipulate and summarize data using a backend library such as Pandas.

Visualize: Create plots and graphs of the data using a graphing library such as Bokeh.

Interact: Accept user input using a frontend library such as React.

Serve: Listen for user requests and return webpages using a web server such as Flask.





Why Streamlit?

Setting Up Streamlit & First App

Before Streamlit, people typically used **web development frameworks** like Flask, Django, or Node.js to create web applications for machine learning. These frameworks provide a lot of flexibility and customizability, but they also require more programming knowledge and time to build an application from scratch.

Other alternatives include using off-the-shelf **dashboard tools** like Tableau, Power BI, Excel for data visualization or using data visualization libraries like Bokeh, Plotly, and D3.js. However, these tools often require more effort to integrate with machine learning models and are not as easily scalable as web-based applications.

Streamlit was created to address the specific needs of machine learning developers, making it easier **to build interactive web applications** for machine learning with minimal coding. It simplifies the process of creating and deploying machine learning applications, allowing developers to focus on building the models and providing a user-friendly interface for non-technical users.

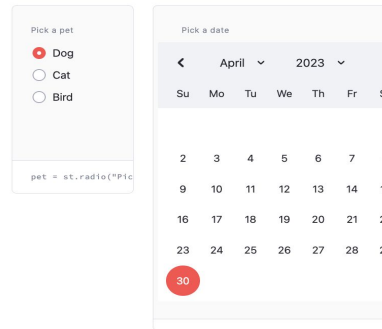
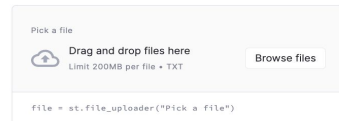


Features

Setting Up Streamlit & First App

Streamlit provides a simple and intuitive interface for creating data-driven applications that can be easily shared and deployed on the web. Its features and advantages include:

- User-friendly and intuitive Python syntax
- Automatic widget generation
- Wide range of data visualization tools and libraries
- Interactive user interfaces for data exploration and analysis
- Easy deployment and hosting options
- Active community support





Getting Started

Setting Up Streamlit & First App

1

Install Streamlit

```
pip install streamlit
```

Terminal

2

Write app

```
import streamlit as st  
  
st.write("Hello, World!")
```

app.py

3

Run app

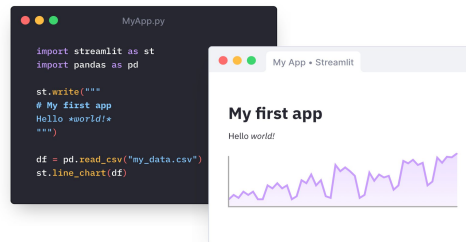
```
streamlit run <filename>.py
```

Terminal

4

View app

Build an app in a few lines of code with simple Streamlit API. Source file automatically updates at every iteration and saves.





Running the App Locally

Setting Up Streamlit & First App

- The Streamlit server runs locally.
- Open the provided local URL in a browser.
- Modify the code and refresh to see changes.



Coding Exercise

Setting Up Streamlit & First App

- Write a Python function that calculates the sum of a list.
- Modify it to take user input using `input()`.
- Create a Streamlit app that:
 - Takes a user's name (`st.text_input()`).
 - Displays a greeting (`st.write()`).



Building Streamlit Apps

Introduction to Python & Streamlit for Beginners

☐ Introduction to Streamlit Components

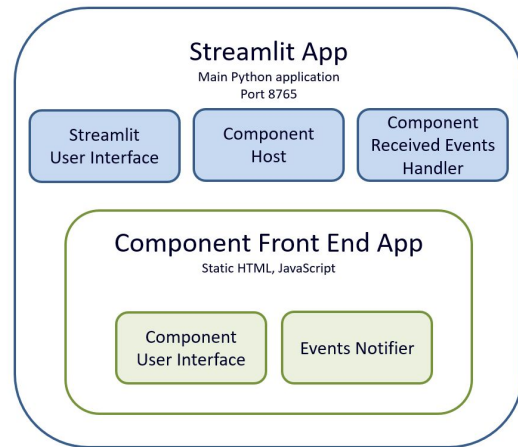
- ☐ Core Streamlit Components
- ☐ Displaying Data in Streamlit
- ☐ Adding Interactivity
- ☐ Coding Exercise: BMI Calculator
- ☐ Structuring a Streamlit App



Components in Streamlit

Introduction to Streamlit Components

- Streamlit provides an easy way to build interactive web apps.
- Components allow user interaction and data visualization.
- We will cover UI elements, data display, and interactivity.



✓ Introduction to Streamlit Components

☐ **Core Streamlit Components**

☐ Displaying Data in Streamlit

☐ Adding Interactivity

☐ Coding Exercise: BMI Calculator

☐ Structuring a Streamlit App



Basic UI Elements in Streamlit

Core Streamlit Components

- `st.text_input()`: Accepts user text input.
- `st.number_input()`: Accepts numerical input.
- `st.button()`: Adds a clickable button.

`st.text_input()`

Value:

`st.number_input()`

Value: 0

`st.button()`

Click me

Status: Not clicked

```
name = st.text_input("Enter your name:")
age = st.number_input("Enter your age:", min_value=0)
if st.button("Submit"):
    st.write(f"Hello {name}, you are {age} years old!")
```



More UI Elements

Core Streamlit Components

- `st.slider()`: Allows selection of a numerical range.
- `st.checkbox()`: Toggle-based input.
- `st.radio()`: Single selection from multiple options.

`st.slider()`



Value: 50

`st.checkbox()`

☐ I agree to terms

Value: Unchecked

`st.radio()`

☒ Option 1

☐ Option 2

☐ Option 3

Selected: option1

```
option = st.radio("Select your favorite fruit:", ["Apple", "Banana", "Cherry"])
st.write(f"You selected: {option}")
```

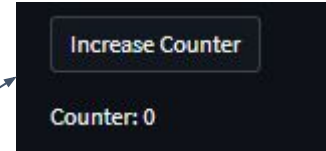


Using `st.session_state` for Interactivity

Core Streamlit Components

- Maintains state across user interactions.
- Example: Storing a counter value across reruns.

```
if 'count' not in st.session_state:  
    st.session_state.count = 0  
if st.button("Increase Counter"):  
    st.session_state.count += 1  
st.write(f"Counter: {st.session_state.count}")
```



✓ Introduction to Streamlit Components

✓ Core Streamlit Components

☐ **Displaying Data in Streamlit**

☐ Adding Interactivity

☐ Coding Exercise: BMI Calculator

☐ Structuring a Streamlit App



Displaying Data in Streamlit

Displaying Data in Streamlit

- `st.write()`: Displays text, data, or plots.
- `st.markdown()`: Supports Markdown formatting.

```
st.write("This is a normal text")  
st.markdown("**This is bold markdown text**")
```

`st.write()`

Simple text output
DataFrame: A B C 0 1 4 7 1 2 5 8 2 3 6 9
`fig = px.line(df, x="A", y="B")`

`st.markdown()`

Title
Subtitle
Regular paragraph with **bold** and *italic* text.

- Bullet point 1
- Bullet point 2

`code block example`



Uploading and Displaying Files

Displaying Data in Streamlit

- `st.file_uploader()`: Allows users to upload files.
- Example: Uploading a CSV file and displaying its contents.

```
uploaded_file = st.file_uploader("Upload a CSV", type=["csv"])
if uploaded_file:
    df = pd.read_csv(uploaded_file)
    st.dataframe(df)
```



Displaying Structured Data

Displaying Data in Streamlit

- Lists & Dictionaries: `st.write()`
- Tables: `st.table()`
- DataFrames: `st.dataframe()` (sortable, filterable)

```
import pandas as pd
data = pd.DataFrame({'Name': ['Alice', 'Bob'], 'Age': [25, 30]})
st.dataframe(data)
```

`st.write()` - Lists & Dictionaries

```
List:
[
  "apple",
  "banana",
  "cherry"
]

Dictionary:
{
  "name": "John Doe",
  "age": 30,
  "city": "San Francisco"
}
```

`st.table()`

name	age	role
Alice	25	Developer
Bob	30	Designer
Charlie	35	Manager



✓ Introduction to Streamlit Components

✓ Core Streamlit Components

✓ Displaying Data in Streamlit

☐ **Adding Interactivity**

☐ Coding Exercise: BMI Calculator

☐ Structuring a Streamlit App

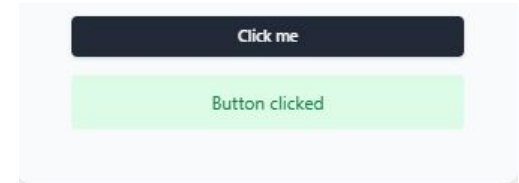


Adding Interactivity

Adding Interactivity

- Dynamically update UI based on input.
- Example: Show a custom message when a button is clicked.

```
if st.button("Click Me"):  
    st.write("Button Clicked!")
```



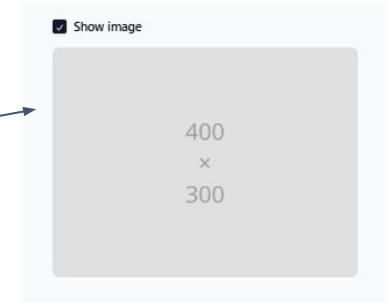


Conditional Rendering

Adding Interactivity

- Show/hide elements based on user selection.
- Example: Display an image only when a checkbox is selected.

```
show_img = st.checkbox("Show Image")  
if show_img:  
    st.image("example.jpg")
```





Creating Forms

Adding Interactivity

- `st.form()`: Groups multiple inputs into a single submission event.
- Example: Collecting user details in a structured form.

```
with st.form("user_form"):  
    name = st.text_input("Name")  
    age = st.number_input("Age", min_value=0)  
    submitted = st.form_submit_button("Submit")  
    if submitted:  
        st.write(f"Hello {name}, you are {age} years old!")
```

Name

Age

Submit

✓ Introduction to Streamlit Components

✓ Core Streamlit Components

✓ Displaying Data in Streamlit

✓ Adding Interactivity

❑ **Coding Exercise: BMI Calculator**

❑ Structuring a Streamlit App



Coding Exercise - BMI Calculator App

Introduction to Streamlit Components

Steps:

1. Use `st.number_input()` to get weight and height.
2. Calculate BMI: $BMI = weight / (height^2)$.
3. Display BMI and categorize:
 - **Underweight:** $BMI < 18.5$
 - **Normal:** $18.5 \leq BMI < 24.9$
 - **Overweight:** $BMI \geq 25$
4. Change the displayed message based on the BMI category.

```
weight = st.number_input("Enter your weight (kg)", min_value=1.0)
height = st.number_input("Enter your height (m)", min_value=0.5)
if st.button("Calculate BMI"):
    bmi = weight / (height ** 2)
    if bmi < 18.5:
        category = "Underweight"
    elif bmi < 24.9:
        category = "Normal"
    else:
        category = "Overweight"
    st.write(f"Your BMI is {bmi:.2f}. Category: {category}")
```

- ✓ Introduction to Streamlit Components
- ✓ Core Streamlit Components
- ✓ Displaying Data in Streamlit
- ✓ Adding Interactivity
- ✓ Coding Exercise: BMI Calculator
- ❑ **Structuring a Streamlit App**



Structuring a Streamlit App

Structuring a Streamlit App

- Well-structured code improves readability and reusability.
- Use functions to break code into logical components.
- Organize UI elements and logic separately
- Use `st.sidebar()` for navigation.



Organizing Code into Functions

Structuring a Streamlit App

- Helps in maintaining and scaling the application.
- Example of separating logic:

```
import streamlit as st

def get_user_input():
    return st.text_input("Enter your name:")

def display_greeting(name):
    st.write(f"Hello, {name}!")

name = get_user_input()
display_greeting(name)
```

Enter your name:

salman

Hello, salman!



Writing Reusable Components

Structuring a Streamlit App

- Create modular UI components.
- Example: Function to generate an input form.

```
def user_form():  
    name = st.text_input("Enter your name")  
    age = st.number_input("Enter your age", min_value=0)  
    return name, age  
  
name, age = user_form()  
st.write(f"User: {name}, Age: {age}")
```

Enter your name

salman

Enter your age

18

User: salman, Age: 18



Using `st.sidebar()` for Navigation

Structuring a Streamlit App

- `st.sidebar()` helps create multi-page apps.
- Example: Selecting different sections.

```
page = st.sidebar.selectbox("Choose a page:", ["Home", "About"])  
if page == "Home":  
    st.write("Welcome to Home Page!")  
elif page == "About":  
    st.write("About this app..")
```

